

Infrastructure Requirements — Dashboard Module

(Standalone On-Premises Deployment)

1. Minimum vs Recommended Hardware

Component	Minimum	Recommended
CPU	Intel Core i5 (4-core, 2.0 GHz+)	Intel Core i7 / Xeon (8-core, 3.0 GHz+)
RAM	8 GB DDR4	32 GB DDR4/DDR5
GPU	None (CPU-only mode, LLM features disabled)	NVIDIA GPU with 7 GB+ VRAM (for Ollama LLM)
GPU VRAM	N/A	8 GB+ (for qwen2.5:7b model)
System Disk (OS + App)	60 GB SSD	200 GB NVMe SSD
Data / Model Disk	N/A (no GPU)	100 GB NVMe SSD (Ollama models + Qdrant data)
Network	100 Mbps (to ServiceNow)	1 Gbps

Note: The Dashboard backend is CPU-only (FastAPI + Pandas). GPU is only required if you want AI-powered leadership insights and automation enrichment via Ollama. All LLM features gracefully degrade to rule-based heuristics if no GPU/Ollama is available.

2. Operating System

Requirement	Specification
OS	Windows Server 2019 / 2022 (recommended) or Ubuntu 22.04 LTS
Architecture	x86-64 (64-bit) only
IIS	IIS 10.0+ with URL Rewrite Module 2.1 (Windows)
PowerShell	PowerShell 5.1+ (for service management)
GPU Drivers	NVIDIA Driver 525+ (only if using Ollama with GPU)
CUDA Toolkit	CUDA 12.1+ (only if using Ollama with GPU)

3. Software Runtime Stack

3.1 Python Backend

Package	Version	Purpose
Python	3.12	Runtime (confirmed in <code>.venv/pyvenv.cfg</code>)
FastAPI	≥ 0.111.0	REST API framework
Uvicorn	≥ 0.29.0 (with standard extras)	ASGI server
httpx	≥ 0.27.0	Async HTTP client (ServiceNow + Ollama calls)
pandas	≥ 2.0.0	In-memory data analysis (incident/change/SR DataFrames)
numpy	≥ 1.26.0	Statistical calculations (SLA%, MTTR, percentiles)
matplotlib	≥ 3.8.0	Server-side PNG chart rendering (Agg backend)
openpyxl	≥ 3.1.0	XLSX fallback data loading
Pillow	≥ 10.3.0	Image processing
pydantic	≥ 2.7.0	Data validation
pydantic-settings	≥ 2.2.2	Settings management from <code>.env</code>
apscheduler	≥ 3.10.0	Background auto-sync scheduler (every 10 min)
qdrant-client	≥ 1.9.0	Vector store client (alert persistence)
python-dotenv	≥ 1.0.0	Environment variable loading

Estimated Python venv disk usage: ~500 MB
No ML/GPU libraries: No PyTorch, TensorFlow, or CUDA libraries in backend.

3.2 Node.js / Frontend Build

Component	Version	Purpose
Node.js	18 LTS or 20 LTS	Frontend build runtime
npm	9+	Package manager
Vite	5.2.13	Build bundler
React	18.3.1	UI framework

Frontend `node_modules` disk usage: ~800 MB–1 GB (build-time only)
Production build output (`dist/`): ~400–600 KB (served as static files)

3.3 Ollama (Optional — LLM Inference Engine)

Component	Specification
Ollama	v0.3.0 or later

Component	Specification
Endpoint	<code>http://localhost:11434</code> (configurable via <code>OLLAMA_BASE_URL</code>)
Default model	<code>qwen2.5:7b</code> (fallback: <code>mistral</code> , <code>llama3.1:8b</code>)
GPU options	<code>num_ctx=2048</code> , <code>num_batch=512</code> , <code>num_gpu=99</code>
Model storage	<code>C:\Users\<user>\.ollama\models\</code> (Windows) or <code>/usr/share/ollama/.ollama/models/</code> (Linux)
Timeout	120 seconds per request (configurable)

If `OLLAMA_ENABLED=false` is set in `.env`, all LLM features are disabled and the module runs fully on rule-based logic with no GPU requirement.

3.4 Qdrant (Optional — Alert Persistence)

Component	Specification
Qdrant	Latest stable (Docker or standalone binary)
Endpoint	<code>http://localhost:6333</code> (configurable via <code>QDRANT_URL</code>)
Collection	<code>dashboard_critical_alerts</code>
Purpose	Persists critical/invoked incidents across backend restarts
Failure mode	If unavailable, critical incidents live in-memory only (lost on restart)

Qdrant is optional. Set `QDRANT_ENABLED=false` to skip it entirely — no functionality is lost except alert persistence across restarts.

4. External Service Dependencies

Service	Type	Required?	Purpose	Failure Mode
ServiceNow	Cloud REST API (HTTPS)	Required for live data	Incident / Change / SR data source	Falls back to XLSX file; shows "Data not synced"
Ollama	Local GPU service	Optional	AI insights, automation enrichment	Degrades to rule-based heuristics
Qdrant	Local vector DB	Optional	Critical alert persistence	Alerts stay in-memory only (lost on restart)

ServiceNow connectivity: The server must have outbound HTTPS access to the ServiceNow instance URL (e.g., `https://<instance>.service-now.com`). This is the only external internet dependency at runtime.

5. LLM Model Requirements (Ollama — Optional)

Priority	Model	VRAM Required	Disk Size	Quality
1 (Default)	qwen2.5:7b	~4–5 GB	~4.7 GB	Good
2 (Fallback)	mistral:7b-instruct	~4 GB	~4.1 GB	Good
3 (Fallback)	llama3.1:8b	~5 GB	~4.7 GB	Good

Pull command: `ollama pull qwen2.5:7b`

LLM inference parameters (configured in `ollama_service.py`):

Parameter	Value
Context window	2,048 tokens
Batch size	512 tokens
GPU layers	99 (all layers to VRAM)
Timeout	120 seconds
Enabled by default	Yes (<code>OLLAMA_ENABLED=true</code>)

LLM use cases in Dashboard:

- `/api/insights` — Executive leadership summary analysis
- `/api/automation-candidates` — Automation type prediction, risk, next steps
- Background pre-warm — Cache LLM results after sync to eliminate first-request latency

6. Data Architecture

The Dashboard is **stateless by design** — no relational database is required.

Data Layer	Technology	Size Estimate	Notes
Primary data	ServiceNow REST API	N/A (streamed)	Paginated, 1,000 records/call
In-memory cache	Pandas DataFrames (3 tables)	50–300 MB	incidents_df, changes_df, service_requests_df; thread-safe with locks
Offline fallback	XLSX file	~17 KB (sample)	Dashboard/Data/FRM_Final_23June2026_TechM - End State.xlsx
Alert persistence	Qdrant (optional)	~10–50 MB	Persists critical incidents across restarts

RAM planning: With 10K incidents + 5K changes + 20K service requests, expect ~100 MB for DataFrames. With 50K+ records, plan for 300+ MB.

7. Disk Storage Breakdown

Component	Estimated Size	Notes
OS (Windows Server 2022)	~30 GB	Base OS
NVIDIA Drivers + CUDA	~5–8 GB	Only if using Ollama with GPU
Ollama binary	~100 MB	Optional inference engine
LLM Model (qwen2.5:7b)	~5 GB	Optional; stored in Ollama model dir
Qdrant binary / Docker image	~200 MB	Optional; data stored in Qdrant data dir
Qdrant data	~50–500 MB	Grows with alert history
Python .venv	~500 MB	Backend dependencies
Node node_modules	~1 GB	Build-time only; can be deleted post-build
Frontend dist/	~5 MB	Served static files
Application code	~10 MB	Source + XLSX fallback
Logs	~200 MB	Uvicorn + scheduler logs
Total (without GPU)	~65–80 GB	CPU-only deployment
Total (with GPU + Ollama)	~80–100 GB	Full feature deployment

Recommended disk: 200 GB NVMe SSD to accommodate OS, Ollama models, Qdrant data, and log growth.

8. Network Requirements

Requirement	Specification
Internal LAN	100 Mbps minimum
Internet (ServiceNow)	Outbound HTTPS (port 443) to ServiceNow instance — required at runtime for sync
Internet (setup only)	Required for pip install, npm install, Ollama model downloads
Internet (runtime)	Outbound HTTPS to ServiceNow only; all other components are local
Firewall — inbound	TCP 8087 (backend API), TCP 8090 (IIS/Nginx frontend)
Firewall — localhost	TCP 11434 (Ollama), TCP 6333 (Qdrant) — must NOT be exposed externally

Requirement	Specification
SSL	<code>SERVICENOW_VERIFY_SSL=false</code> is the current default (set to <code>true</code> in production with valid cert)

9. Port Reference

Service	Port	Protocol	Scope
Dashboard Backend (Uvicorn)	8087	HTTP	Internal / IIS-proxied
Dashboard Frontend (dev)	5178	HTTP	Dev only
Ollama LLM	11434	HTTP	Localhost only
Qdrant Vector Store	6333	HTTP	Localhost only
IIS (production)	8090	HTTP	Client-facing
ServiceNow	443	HTTPS	Outbound to cloud

10. Environment Variables

Create a `.env` file at `Dashboard/.env`:

```
# Logging
PYTHONUNBUFFERED=1
LOG_LEVEL=INFO

# ServiceNow (primary data source)
SERVICENOW_BASE_URL=https://<instance>.service-now.com
SERVICENOW_USERNAME=<servicenow-username>
SERVICENOW_PASSWORD=<servicenow-password>
SERVICENOW_VERIFY_SSL=true
SERVICENOW_TIMEOUT_SECONDS=20

# Auto-sync interval
AUTO_SYNC_INTERVAL_MINUTES=10

# CORS (add all origins that will access the backend)
CORS_ORIGINS=http://localhost:8090,http://<server-ip>:8090

# Ollama LLM (set OLLAMA_ENABLED=false to disable AI features entirely)
OLLAMA_ENABLED=true
OLLAMA_MODEL=qwen2.5:7b
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_TIMEOUT_SECONDS=120

# Qdrant vector store (set QDRANT_ENABLED=false to disable alert persistence)
QDRANT_ENABLED=true
```

```
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=dashboard_critical_alerts
```

Create `Dashboard/frontend/.env.production`:

```
VITE_DASH_API_URL=http://<server-ip>:8087/api
```

11. IIS Configuration (Windows On-Prem)

Component	Requirement
IIS Version	10.0+
URL Rewrite Module	2.1+
ARR (Application Request Routing)	Required for reverse proxy to backend
Static file handler	For serving <code>frontend/dist/</code>
SPA fallback rule	Rewrite all non-API <code>/dash/*</code> routes to <code>index.html</code>

URL Rewrite rule (in `web.config`):

```
<!-- Proxy API calls to FastAPI backend -->
<rule name="Dashboard API Proxy" stopProcessing="true">
  <match url="^api/dashboard/(.*)" />
  <action type="Rewrite" url="http://localhost:8087/api/{R:1}" />
</rule>

<!-- SPA fallback for React Router -->
<rule name="Dashboard SPA Fallback" stopProcessing="true">
  <match url="^dash/(.*)" />
  <conditions>
    <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
  </conditions>
  <action type="Rewrite" url="/dash/index.html" />
</rule>
```

12. Service Management

Component	Method
Backend startup	<code>python main.py</code> or <code>uvicorn main:app --host 0.0.0.0 --port 8087</code>
Auto-restart on crash	Register via Windows Task Scheduler (at logon, highest privilege)
Ollama startup	<code>ollama serve</code> (register as Windows Service or Task Scheduler entry)

Component	Method
Qdrant startup	<code>docker run -d -p 6333:6333 qdrant/qdrant</code> or standalone binary
Log location	Uvicorn stdout / stderr (redirect to file via Task Scheduler)

Start command (manual):

```
# Start Qdrant (optional)
docker run -d -p 6333:6333 -p 6334:6334 qdrant/qdrant

# Start Ollama (optional)
Start-Process "ollama" -ArgumentList "serve" -NoNewWindow

# Start Dashboard backend
cd C:\STIQ\StratIQ_VM_AWS\Dashboard\backend
.\.venv\Scripts\python.exe -m uvicorn main:app --host 0.0.0.0 --port 8087
```

13. Capacity Planning

Workload	Recommended Config
Small org (<10K records, 1–5 users)	8 GB RAM, 4-core CPU, no GPU (CPU-only mode)
Medium org (10K–50K records, 5–20 users)	16 GB RAM, 8-core CPU, optional 7 GB VRAM GPU
Large org (50K+ records, 20+ users)	32 GB RAM, 16-core CPU, NVIDIA GPU (8+ GB VRAM)

Throughput notes:

- Initial ServiceNow sync: ~1–5 seconds (100 records/sec paginated)
- Auto-sync interval: every 10 minutes (configurable)
- Ollama insight generation: ~10–30 seconds per request
- Data cache: refreshes from ServiceNow; no disk I/O during queries

14. Pre-Deployment Checklist

Infrastructure

- ☐ Server provisioned with Windows Server 2022 or Ubuntu 22.04 LTS
- ☐ Python 3.12 installed (`python --version` confirms 3.12.x)
- ☐ Node.js 18 LTS or 20 LTS installed (for frontend build only)
- ☐ Outbound HTTPS (port 443) access to ServiceNow instance confirmed
- ☐ Firewall: TCP 8087 open to internal LAN; TCP 11434 and 6333 restricted to localhost

Python Backend

- ☐ Python 3.12 venv created: `python -m venv Dashboard\backend\.venv`

- ☐ Dependencies installed: `pip install -r Dashboard\backend\requirements.txt`
- ☐ `.env` file created at `Dashboard\.env` with ServiceNow credentials
- ☐ Backend starts: `python Dashboard\backend\main.py` → verify at `http://localhost:8087/api/status`

Frontend

- ☐ `npm install` run in `Dashboard\frontend\`
- ☐ `Dashboard\frontend\.env.production` updated with server IP
- ☐ `npm run build` run → confirms `frontend\dist\` created
- ☐ `dist\` served via IIS or Nginx

Optional: Ollama (LLM Features)

- ☐ NVIDIA GPU driver (525+) installed and verified (`nvidia-smi`)
- ☐ Ollama installed and `ollama serve` running on port 11434
- ☐ Model pulled: `ollama pull qwen2.5:7b`
- ☐ `OLLAMA_ENABLED=true` and `OLLAMA_BASE_URL` set in `.env`

Optional: Qdrant (Alert Persistence)

- ☐ Docker installed (or Qdrant standalone binary)
- ☐ Qdrant started: `docker run -d -p 6333:6333 qdrant/qdrant`
- ☐ `QDRANT_ENABLED=true` and `QDRANT_URL` set in `.env`

Post-Deployment Verification

- ☐ Navigate to `http://<server-ip>:8090/dash/` — login page appears
- ☐ Enter ServiceNow credentials and click Connect
- ☐ Click Sync — KPI cards and charts populate
- ☐ Executive Cockpit loads with incident/change/SR data
- ☐ (Optional) Insights tab returns AI-generated summary

15. Summary — Minimum Viable On-Prem Deployment

Item	CPU-Only (No AI)	Full (with AI Insights)
CPU	4-core i5	8-core i7 / Xeon
RAM	8 GB	32 GB
GPU	None	NVIDIA 8 GB+ VRAM
Storage	60 GB SSD	200 GB NVMe SSD
OS	Windows Server 2022 or Ubuntu 22.04	Same
Python	3.12	3.12
Node.js	18 LTS (build only)	18 LTS (build only)

Item	CPU-Only (No AI)	Full (with AI Insights)
Ollama	Not required	Required (qwen2.5:7b)
Qdrant	Optional	Optional (recommended)
ServiceNow	Outbound HTTPS required	Outbound HTTPS required
External cloud	ServiceNow only	ServiceNow only